

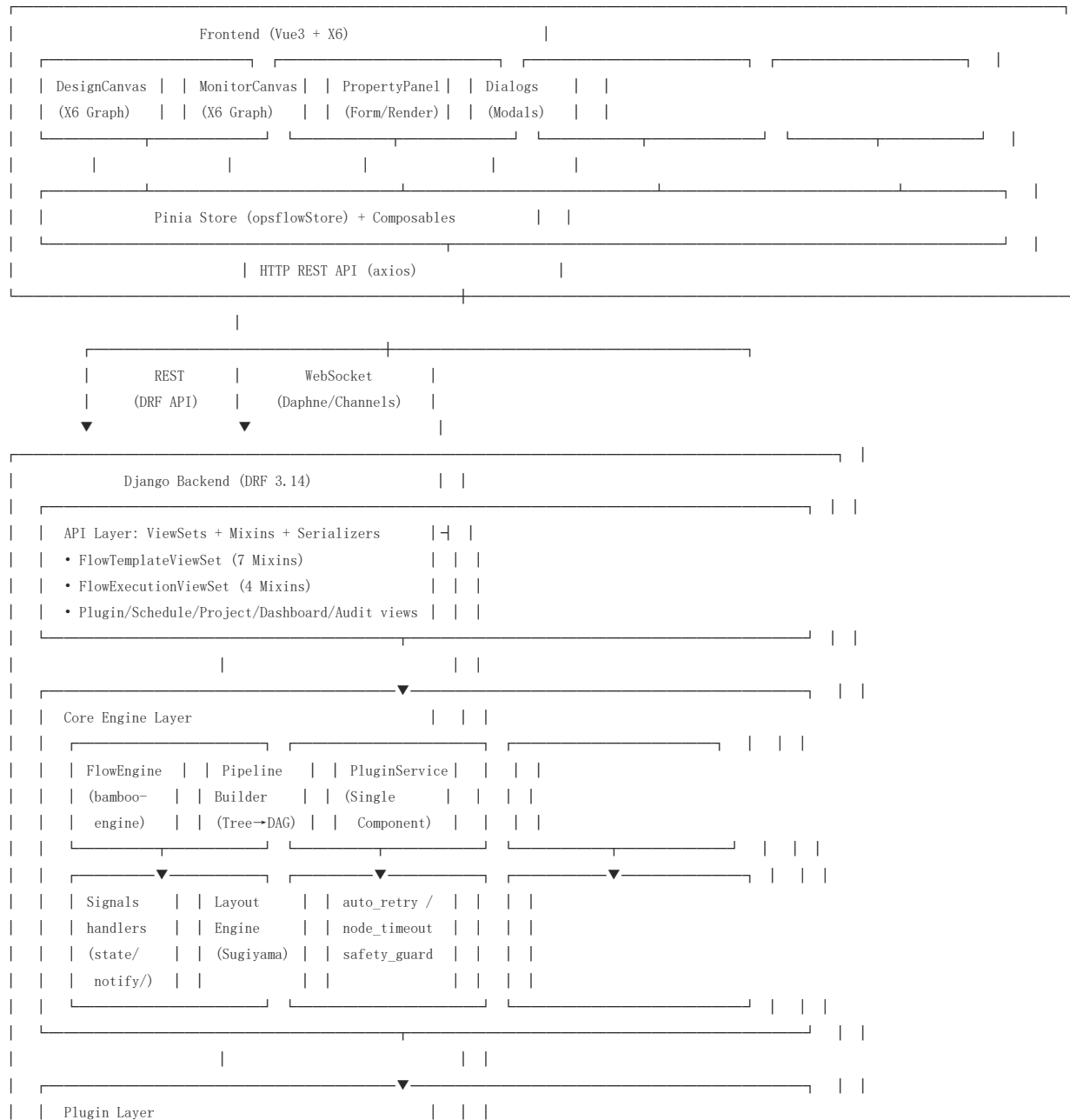
OpsFlow 系统架构设计

系统架构 — 分层设计 + 执行流程 + 关键决策

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

1. 整体架构





2. 五层设计

2.1 Frontend 层 (Vue3 + TypeScript + X6)

模块	说明
`DesignCanvas.vue`	基于 AntV X6 的流程设计器画布
`MonitorCanvas.vue`	执行监控画布（节点着色 + 状态标记）
`PropertyPanel.vue`	右侧属性面板（通用 RenderForm）
`PluginPickerDialog.vue`	插件选择器弹窗
`opsflowStore.ts`	Pinia 状态管理（模板/执行/项目/变量）
`useDesignCanvas.ts`	画布交互组合函数
`useMonitor.ts`	监控实时刷新组合函数
`useAutoSave.ts`	模板自动保存组合函数
`utils/shapes.ts`	X6 自定义图形注册

2.2 API 层 (Django REST Framework)

ViewSet	端点	功能
FlowTemplateViewSet	`/api/opsflow/templates/`	模板 CRUD + 7 个 Mixin
FlowExecutionViewSet	`/api/opsflow/executions/`	执行 CRUD + 4 个 Mixin
SchedulePlanViewSet	`/api/opsflow/schedule-plans/`	定时计划
PluginViewSet	`/api/opsflow/plugins/`	插件管理
OpsProjectViewSet	`/api/opsflow/projects/`	项目管理
Dashboard views	`/api/opsflow/dashboard/`	仪表盘统计

2.3 Core Engine 层

核心调度层，基于 [bamboo-engine] + [BambooDjangoRuntime] 驱动：

模块	职责
`FlowEngine`	流程执行引擎 (start/pause/resume/retry/skip/cancel/run)
`PipelineBuilder`	将自定义 pipeline_tree 转换为 bamboo-engine 可执行 DAG
`PluginService`	单一 Service 组件，运行时分发到 PLUGIN_REGISTRY
`Signals`	状态变更信号处理 (state/notify/trace/timeout)
`Layout Engine`	Sugiyama 分层布局算法 (acyclic/rank/order/position)
`Safety Guard`	Pipeline Tree 安全校验 (白名单/重试上限/循环引用)
`AutoRetry`	节点自动重试策略管理
`NodeTimeout`	Redis 分布式超时追踪 + 超时策略执行
`NodeDispatcher`	节点操作调度器 (retry/skip/force_fail + Trace 记录)
`LLM Service`	AI 流程生成 (DeepSeek API + RAG)
`SubprocessDispatcher`	独立子流程调度
`WebhookService`	Webhook 回调分发 (HMAC 签名 + 重试)

2.4 Plugin 层

插件自动注册体系：

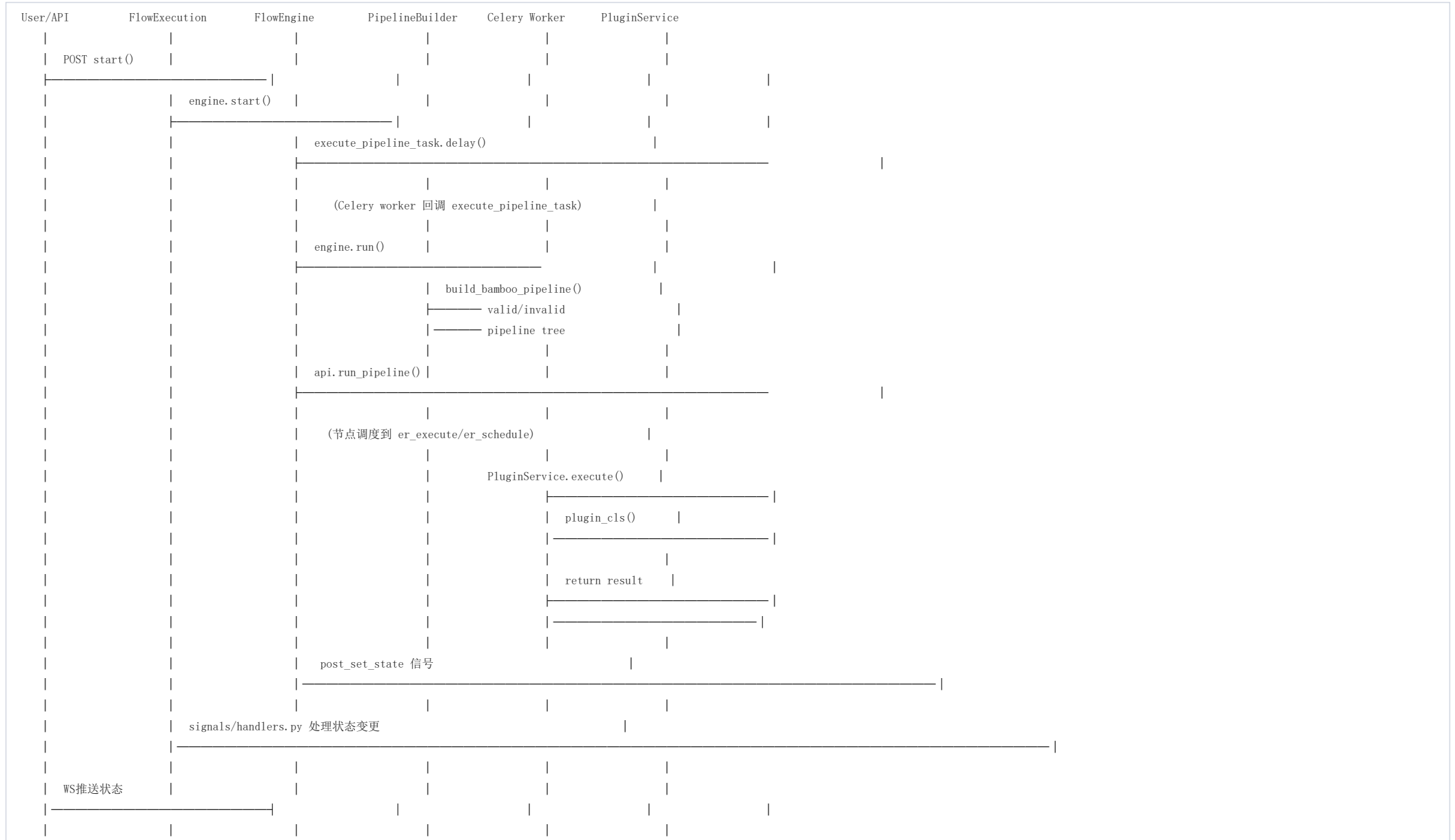
```
opsflow/plugins/
├── base.py          # BasePlugin 基类
├── registry.py      # 自动扫描注册器 (PLUGIN_REGISTRY)
├── ansible/         # shell, file_copy, java_deploy, docker_deploy...
├── common/          # send_alert, test_print_time
├── esxi/            # create_vm, power_on/off, destroy_vm...
├── http/            # api_call
├── monitor/         # disk_check, health_check, ping_test
├── netapp/          # volume/snapshot operations
├── pmax/            # storage_group, snapshot, performance
├── redfish/         # firmware_inventory, system_info, power_cycle
└── servicenow/      # ITSM integration
```

每个插件继承 [BasePlugin]，实现 [execute()]，[get_form_config()]，可选实现 [schedule()]，[rollback()]。

2.5 Data 层

存储	用途
MySQL	所有业务模型（模板/执行/插件/项目/审计日志）
Redis	节点超时追踪（Sorted Set `opsflow:executing_nodes`）、幂等锁、缓存
文件系统	节点轨迹日志（JSON Lines，路径 `{LOG_DIR}/opsflow/tasks/{id}/`）
Celery	异步任务队列（er_execute/er_schedule/default）

3. 执行流程时序



4. 关键设计决策

4.1 BambooDjangoRuntime 迁移

从 [atom_service.py]（按原子类型动态创建 N 个 Component/Service）迁移到统一的 [PluginService] + [BambooDjangoRuntime] 模式。所有原子通过一个 OpsflowPluginComponent 注册，运行时从 [PLUGIN_REGISTRY] 查找。

****核心变更：****

- 删除旧的 [atom_
- 将 [Component
- [PluginServic
- 执行完全走 [b

4.2 Executor Factory (NodeCommandDispatcher)

替代 [executor_factory.py] 中的命令模式，[NodeCommandDispatcher] 封装所有节点操作：

NodeCommandDispatcher		
└─ retry(node_id, operator)	→	创建 Trace + 调用 FlowEngine.retry()
└─ skip(node_id, operator)	→	调用 FlowEngine.skip()
└─ force_fail(node_id, reason)	→	调用 FlowEngine.force_fail()
└─ get_trace(node_id)	→	查询 NodeExecutionTrace
└─ get_all_traces_summary()	→	所有节点轨迹摘要

4.3 Component 注册

从多 Component 模式简化为单 Component 模式：

```
class OpsflowPluginComponent(Component):
    name = "OpsFlow Plugin"
    code = "opsflow_plugin"
    bound_service = PluginService
```

[PluginService.execute()] 从 [data.inputs] 提取 [_atom_type] 参数，查询 [PLUGIN_REGISTRY] 获取对应的插件类并调用其 [execute()] 方法。变量解析通过 [variable_resolver.resolve_params()] 完成 [{key}] 替换。

4.4 AI 集成

基于 DeepSeek API 的 AI 流程生成：

- `**[generate_pipeline()]**`: 自然语言 → Pipeline Tree JSON。内置 RAG 检索 `[OpsKnowledge]` 作为示例注入。
- `**[refine_pipeline()]**`: 多轮对话修改现有流程。区分“问答”和“修改”两种模式。
- `**[analyze_pipeline()]**`: 分析流程的目的、步骤和潜在风险。
- `**校验管线**`: AI 输出 → `[validate_pipeline()]` (白名单/结构/安全) → `[validate_bamboo_compatibility()]` → 自动增加 `[group]/[risk_level]` 富化。

4.5 Layout 引擎

替代 `bk_sops` 的 `[pipeline_web]` 布局模块，使用纯 Python Sugiyama 分层布局算法：

```
compute_layout(nodes, edges)
→ opsflow_to_pipeline()      格式转换
→ draw_pipeline()           Sugiyama 五步法:
    1. acyclic (反转环边)
    2. rank (最长路径层级分配 + 紧树压缩)
    3. order (交叉最小化: WDA 算法)
    4. position (坐标分配: 居中 + Py)
    5. drawing (边线生成)
→ pipeline_to_positions()     返回 [{id, x, y}]
```

4.6 执行隔离

每个 `[FlowExecution]` 在创建时拷贝 `[template_snapshot]` 的副本，实现模板修改与执行实例的完全隔离：

```
# 创建时冻结
execution.template_snapshot = {
    'pipeline_tree': snapshot_tree,
    'target_hosts': template.target_hosts,
    'global_vars': template.global_vars,
    'template_version': template.version,
}
```

4.7 项目隔离

`[ProjectFilteredViewSet]` 基类自动按项目成员关系过滤数据，非超管用户只能访问自己有权限的项目资源。公共模板通过 `[is_public]` + `[project_scope]` 实现跨项目共享。

4.8 API 响应规范

所有 opsflow API 统一使用 [DetailResponse] / [ErrorResponse] / [SuccessResponse] 模板，确保前端 interceptor 能解析 [code] 字段：

场景	响应类	code
成功（单条）	<code>`DetailResponse(data)`</code>	2000
成功（分页）	<code>`get_paginated_response()`</code>	2000
业务错误	<code>`ErrorResponse(msg, code=4000)`</code>	4000
AI 错误	<code>`ErrorResponse(msg, data, code=4000)`</code>	4000